



Guion de Clase

Unidad 02:

Pruebas de Software

Objetivos

Que el estudiante sea capaz de:

- Definir qué son las pruebas de software
- Definir que es el ágil testing
- Definir y aplicar las estrategias de pruebas de software
- Definir y aplicar las pruebas no funciones en los sistemas.

Definición de Pruebas de Software

La prueba es un conjunto de actividades que pueden planearse por adelantado y realizarse de manera sistemática. Por esta razón, durante el proceso de software, debe definirse una plantilla para la prueba del software: un conjunto de pasos que incluyen métodos de prueba y técnicas de diseño de casos de prueba específicos.

En la literatura sobre el tema, se han propuesto algunas estrategias de prueba de software. Todas proporcionan una plantilla para la prueba y tienen las siguientes características genéricas:

- Para realizar una prueba efectiva, debe realizar revisiones técnicas efectivas (Plan de aseguramiento de la calidad)
- La prueba comienza en los componentes y opera “hacia afuera”, hacia la integración de todo el sistema informático.
- Diferentes técnicas de prueba son adecuadas para distintos enfoques de ingeniería de software y en diferentes momentos en el tiempo.
- Prueba y depuración son actividades diferentes, pero la depuración debe incluirse en cualquier estrategia de prueba.

Para tener una definición clara de lo que son las pruebas de Software primero debemos respondernos a la siguiente pregunta ¿Por qué debo de probar un software? Algunas de las respuestas que podemos pensar son:

- **Nos ayuda a reducir costo:** porque sabemos exactamente que queremos revisar durante el desarrollo del proyecto lo que nos ahorraría tiempo programador debido a que tendremos la certeza que está bien diseñada nuestra aplicación antes de iniciar con su proceso de programación



- **Evita retraso en los tiempos estimados:** porque al momento de las revisiones con el usuario sabemos que la aplicación responderá a los requerimientos específico que el usuario nos ha brindado; y centraríamos la atención en la definición correcta de esos requerimientos obtenido en la etapa de definición de los requerimientos.
- **Reduce la insatisfacción de usuario:** porque el software entregado debe cumplir con la necesidad que el usuario tiene y no dará errores que provocarán que el usuario se sienta desconfianza al usar la aplicación.
- **El diseño de pruebas reduce el costo de mantenimiento:** ya que, al momento de diseñar las pruebas, se revisará tanto calidad del producto como del proceso, lo que disminuiría en gran medida el mantenimiento de una aplicación.

Hay una premisa que debe tomarse en cuenta a la hora de realizar pruebas de software y es “El mismo programador pruebe sus desarrollos”.

El motivo es que el programador conoce cuales es la debilidad que puede tener el código; conoce las validaciones, los tipos de datos utilizados y funcionamiento de una pantalla en específico; por lo que ocasiona que no pueda detectar elementos que se pudieron haber quedado fuera y posiblemente no realizara una prueba exhaustiva en busca de algún error.

Es una buena práctica que las pruebas formales de software la realicen un equipo dedicado a esta tarea o alguna otra persona del equipo de desarrollo para que las pruebas sean mucho más eficientes, confiables y exactas.

Los Objetivos que persigue el realizar pruebas de software son los siguientes:

- Realizar las pruebas de validación
- Realizar pruebas de defecto

Las pruebas de validación son las que están relacionadas con el cumplimiento de la totalidad de los requerimientos planteados.

Las pruebas de defecto que son las más difíciles de detectar ya que son errores en la lógica del negocio. Son desarrolladas cuando ya el software tiene un grado de avance significativo y nos permite integrar diferentes componentes de este.

La realización de pruebas de software nos brinda beneficios como:

- Encontrar situaciones donde el comportamiento no sea el esperado para poder corregirlas y evitar que vuelvan a suceder.
- Verificar que cada uno de los componentes estén integrados de tal manera que permitan desarrollar toda la lógica del negocio de manera continua e interrumpida.
- Verificar que se cumplan las necesidades del usuario; tal cual estén descritas en los requerimientos que nos proporcionados.

Según it-Mentor Testing o Pruebas de Software es la tarea de demostrar que un programa realizar funciones para las cuales fue construida.



Existen cinco elementos que hay que tener presente para cualquier prueba de software que se desean realizar:

1. Estrategia de pruebas
2. Plan pruebas
3. Casos de Pruebas
4. Datos de Prueba
5. Ambiente de pruebas

Principios de las pruebas de software

- La prueba puede ser usada para mostrar la presencia de errores, pero nunca su ausencia.
- La principal dificultad del proceso de prueba es decidir cuándo parar.
- Evitar casos de pruebas no planificados, no reusables y triviales a menos que el programa sea verdaderamente sencillo.
- Una parte necesaria de un caso de prueba es la definición del resultado esperado.
- Los casos de pruebas tienen que ser escritos no solo para condiciones de entrada válidas y esperadas sino también para condiciones no válidas e inesperadas.

Agile Testing

Es una práctica de pruebas de software que sigue los mismos principios del desarrollo ágil se enfoca en que todo el equipo de desarrollo debe de ayudar a integrar la calidad del producto a la calidad del proceso que se definió.

Esta técnica basa su implementación en que debe de ser parte del proceso de desarrollo y no ser vista como una etapa al finalizar el software, sino que debe de iniciarse con estas pruebas desde que se inicia con el análisis, diseño y programación.

Esta práctica no solo ve las pruebas de cara al usuario, sino que verifica internamente como se está realizando el proceso de desarrollo para poder evitar inconvenientes futuros.

Principio Agile Testing

Los principios que rigen el Agile testing son los siguientes:

1. **Testing no es una fase:** la etapa de pruebas y revisiones se debe de hacerse en todo momento no hasta finalizar el proyecto
2. **Testing de manera continua:** ya que nos permite avanzar en un proyecto de desarrollo dando resultados más inmediatos y precisos.
3. **Todo el equipo debe de realizar pruebas.** La recomendación es que cada uno pase por la etapa de pruebas; pero no probando lo que él mismo desarrolló, sino que

realizando pruebas del desarrollo de otro compañero del equipo de trabajo; así como también nuestros usuarios se convierta en parte esencial de este proceso pues son los que nos darán una mejor retroalimentación de los diseñado.

4. **Código más limpio:** porque se debe de revisar en todo momento que la calidad del proceso definido se esté cumpliendo a su totalidad. Los errores encontrados se corrigen de manera inmediata y se verifica que los desarrolladores cumplan con el estándar establecido.
5. **Reducir el tiempo para recibir retroalimentación por parte del usuario:** porque los usuarios deberían de estar involucrados en las pruebas y ayudarnos a realizarlas también. Ya que ellos son los que conocen el negocio y saben que es lo que necesitan, estas pruebas deben realizarse con la ayuda de alguien del equipo de desarrollo para orientarlos en el uso del software y para que así nos puedan dar sus sugerencias como se menciona en el principio 4.
6. **Desarrollo guiado por pruebas:** se debe de guiar siempre por pruebas específicas; debemos saber qué es lo que estamos probando y seguir un plan que se define junto con el equipo de desarrollo.

Rol del Tester

En el agile testing el tester se convierte en uno de los roles fundamentales durante el desarrollo del proyecto ya que se convierte en ese experto que nos garantiza que se esté desarrollando lo que el usuario necesita basado en los requerimientos que nos ha brindado. Nos ayuda también a que el equipo de desarrollo realice el trabajo asignado a un ritmo sostenible y continuo.

El Tester trabaja en paralelo con el desarrollo, ya que define como se probará cada una de las pantallas desarrolladas para que todos cumplan con ese objetivo propuesto.

El Tester es el enlace entre el cliente y el desarrollador. Él es el encargado de realizar las pruebas de usuarios junto con el cliente, también es quién transmite las dificultades y aciertos que se ha tenido con las pruebas realizadas con el cliente.

Es quién ayuda a revisar la calidad del producto y del proceso definido por el equipo de desarrollo durante la etapa de análisis y diseño.

En un entorno ágil, el rol del tester es de mayor especialización técnica, considerando que debe manejar herramientas de automatización, gestión ágil y metodologías; además, necesitará de comunicación, orientación al cliente y negociación.

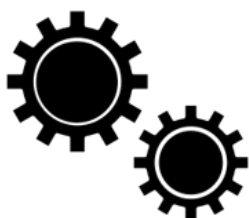
Cuadrantes del Agile Testing

Los cuadrantes fueron introducidos en 2003 por Brian Marick, él define los cuadrantes como una variedad de emparejamientos visto desde diferentes perspectivas tanto del negocio como de parte del equipo de desarrollo. Luego Lisa Crispin y Janet Gregory los adaptaron en su obra "Agile Testing".

Los cuadrantes representan los diferentes propósitos y tipos de pruebas de software que podemos realizar en un entorno ágil. Los cuadrantes se enmarcan como se muestran en la siguiente imagen:



Cuadrante 1 (Q1)



El cuadrante 1 está definido por el **Desarrollo Guiado por Pruebas (TDD)**, que es una práctica que consiste en elegir un requerimiento, al cual se le realizará el análisis y diseño para luego definir como se realizará la prueba de ese requerimiento al mismo tiempo que se está realizando la programación de este. Luego al finalizar la programación se procede a verificar la prueba. Se verifica si la prueba pasa o no, sino pasa se debe de revisar ya sea la programación o la definición de la prueba hasta que se cumpla con éxito. Al finalizar se da por concluido el desarrollo de ese requerimiento y se pasa al siguiente hasta concluir con todos los requerimientos definidos.

Este cuadrante incluye todas aquellas pruebas de software que deben de hacerse a pantallas individuales de todo el software que se desarrolle sin excepción.

El tester puede auxiliarse de algunas herramientas que ofrecen los frameworks de programación para realizar pruebas específicas a pantallas. Estas son denominadas herramientas de pruebas automatizadas. Algunos ejemplos de estas herramientas son PHPUnit, Selenium, Watir por mencionar algunas.

También se realizan pruebas de componentes que son las que nos ayudan a determinar cómo se deben de integrar entre si todas las pantallas o interfaces de la aplicación que se está desarrollando.

Cuadrante 2 (Q2)

Las pruebas de cuadrante 2 son pruebas que pueden realizarse tanto manuales como automatizadas. Ya que el equipo realiza pruebas de alto nivel integrando diferentes elementos y flujos de negocio con la ayuda de herramientas automatizadas; pero al mismo tiempo verifican que cumplan con el objetivo por el cual fueron desarrolladas.



Además, representan las pruebas de cliente desarrolladas por el equipo de desarrollo, determinando si las funcionalidades que se están solicitando en las historias de usuario se cumplen o no. Estas pruebas sirven también para darles una visión del producto al equipo, y así determinar qué es lo que no se está tomando en cuenta de los estándares definidos y si se respetan los criterios de calidad del producto que se plantearon al inicio del proyecto.

Deben de realizarse de manera regular para poder identificar situaciones no deseadas y solucionar errores rápidamente. Entran en juego algunos prototipos de interfaces que es una realización rápida de la aplicación para verificar colores, disposición de elementos, inclusión completa de campos solicitados, etc.

Cuadrante 3 (Q3)



Las pruebas de cuadrante 3 son específicas del cliente. Son aquellas pruebas que se realizan entre el tester y el cliente que nos ha contratado para realizar el desarrollo del software. Es una prueba integral para determinar si se cumple con las expectativas del cliente. Se trata de simular lo más que se pueda el entorno en el cual se ejecutará en producción el producto, y es por este motivo que son pruebas manuales.

Las pruebas de los criterios de usabilidad también son parte de este cuadrante. Generalmente las pruebas que realiza el usuario nos sirven como parámetro para ser tomadas como pruebas de aceptación porque cuentan con la aprobación inmediata del usuario.

En este cuadrante es donde se logran detectar los errores de lógica de negocio. Porque si hay algo que no llegamos a comprender bien durante el desarrollo al ser el usuario el encargado de realizar las pruebas es quien nos hará saber de inmediato que hemos cometido algún error.

Cuadrante 4 (Q4)

En el cuadrante 4, se localizan las pruebas de todos los requerimientos no funcionales definidos en los criterios de calidad del producto, como el rendimiento, tiempos de respuesta y seguridad por mencionar algunos. Generalmente son pruebas que el 100% se realizan con herramientas que ya existen en el mercado como lo es PageSpeed Insights, Latency, Wireshark entre otras.



Se podrían reutilizar pruebas del cuadrante 1, por ejemplo, ejecutar simultáneamente las pruebas unitarias para ver cómo se comporta el sistema.

Se hace una revisión de todos los criterios de calidad del proceso para verificar si se ha realizado de manera correcta con la documentación necesaria para el desarrollo.

Relación entre Cuadrantes

El agile testing se diferencia de técnicas tradicionales ya que al mezclar pruebas entre cuadrantes se va realizando pruebas enfocadas a los diferentes actores del proceso de desarrollo. Las pruebas de los Cuadrantes 1 y 2 deben ejecutarse durante la etapa del desarrollo. Sin dejar de lado ninguno de los componentes antes descritos. Es por esto que estos cuadrantes son denominados de **Soporte al desarrollo**.

Los cuadrantes 3 y 4 representan cuando la programación ha sido finalizada y se requiere de revisiones más exhaustivas de lo desarrollado, es por eso que estas pruebas **evalúan el producto**.

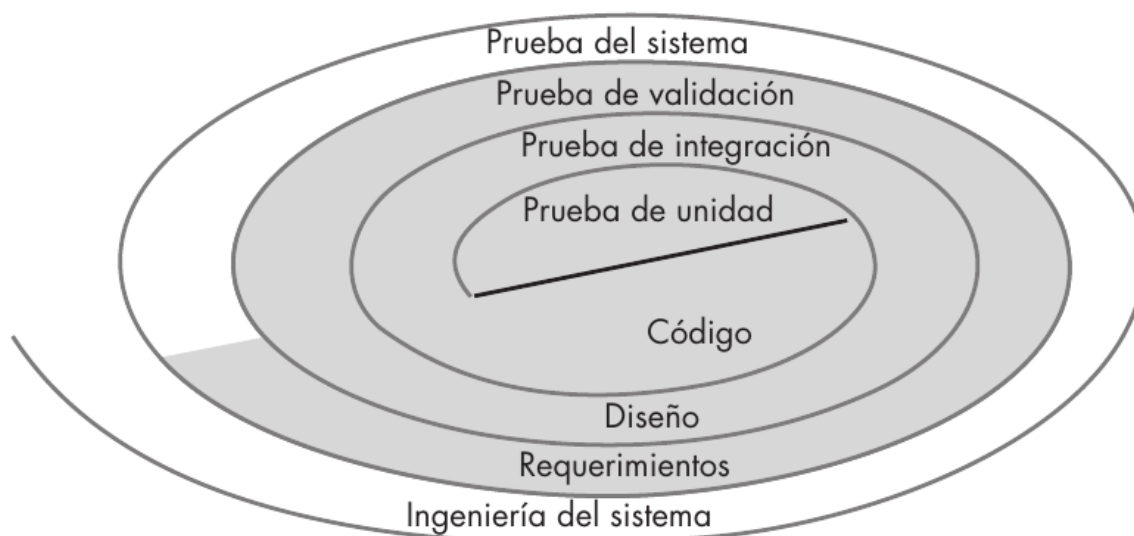
Las pruebas de cuadrante 2 y 3 se realizan de manera conjunta para determinar si la **perspectiva del negocio** se ha cubierto en su totalidad.

Las pruebas de cuadrante 4 y 1 se realizan para verificar que la **perspectiva de la tecnología** se ha realizado de manera correcta, se hayan aplicado los estándares definidos y aplicado los framework de programación de una manera adecuada.

Pruebas de Aplicación Tradicionales

Estrategia de prueba del software.

El proceso de software puede verse como la espiral que se muestra en la siguiente imagen:



La prueba de unidad comienza en el vértice de la espiral y se concentra en cada uno de los requerimientos específicos de la aplicación, cada pantalla del sistema y la relación de cómo fue implementada o programada en el código fuente. A medida la prueba avanza, al moverse hacia afuera a lo largo de la espiral, hacia la prueba de integración, donde el enfoque se centra en el diseño y la construcción de la arquitectura del software. Al dar otra vuelta hacia afuera de la espiral, se encuentra la prueba de validación, donde los requerimientos establecidos como parte de su modelado se validan confrontándose con el software que se construyó. Finalmente, se llega a la prueba del sistema, donde el software y otros elementos del sistema se prueban como un todo. Para probar el software desarrollado se avanza en espiral hacia afuera en dirección de las manecillas del reloj a lo largo de líneas que ensanchan el alcance de las pruebas con cada vuelta.

Existen muchas estrategias que pueden usarse para probar el software. En un extremo, puede esperarse hasta que el sistema esté completamente construido y luego realizar las pruebas sobre el sistema total, con la esperanza de encontrar errores. Este enfoque, aunque atractivo, simplemente no funciona. Dará como resultado software defectuoso que desilusionará a todos los participantes. En el otro extremo, podrían realizarse pruebas diariamente, siempre que se construya alguna parte del sistema. Este enfoque, aunque menos atractivo para muchos, puede ser muy efectivo.



Una estrategia de prueba que eligen la mayoría de los equipos de software se coloca entre los dos extremos. Toma una visión incremental de las pruebas, comenzando con la de unidades de programa individuales, avanza hacia pruebas diseñadas para facilitar la integración de las unidades y culmina con pruebas que ejercitan el sistema construido.

Los tipos de pruebas que se encuentran en un enfoque tradicional son:

- Pruebas de Unidad
- Pruebas de Integración
- Pruebas de Validación
- Pruebas de Sistemas

Pruebas de unidad

La prueba de unidad enfoca los esfuerzos de verificación en la unidad más pequeña del diseño de software: el componente o módulo de software. Al usar la descripción del diseño de componente como guía, las rutas de control importantes se prueban para descubrir errores dentro de la frontera del módulo. La relativa complejidad de las pruebas y los errores que descubren están limitados por el ámbito restringido que se establece para la prueba de unidad. Las pruebas de unidad se enfocan en la lógica de procesamiento interno y de las estructuras de datos dentro de las fronteras de un componente. Este tipo de pruebas puede realizarse en paralelo para múltiples componentes.

Consideraciones de las pruebas de unidad.

La interfaz del módulo se prueba para garantizar que la información fluya de manera adecuada hacia y desde la unidad de software que se está probando. Las estructuras de datos locales se examinan para asegurar que los datos almacenados temporalmente mantienen su integridad durante todos los pasos en la ejecución de un algoritmo. Todas las rutas independientes a través de la estructura de control se ejercitan para asegurar que todos los estatutos en un módulo se ejecuten al menos una vez. Las condiciones de frontera se prueban para asegurar que el módulo opera adecuadamente en las fronteras establecidas para limitar o restringir el procesamiento. Y, finalmente, se ponen a prueba todas las rutas para el manejo de errores.

El flujo de datos a través de la interfaz de un componente se prueba antes de iniciar cualquiera otra prueba. Si los datos no entran y salen de manera adecuada, todas las demás pruebas son irrelevantes. Además, deben ejercitarse las estructuras de datos locales y averiguarse el impacto local sobre los datos globales durante las pruebas de unidad.

La prueba selectiva de las rutas de ejecución es una tarea esencial durante la prueba de unidad. Los casos de prueba deben diseñarse para descubrir errores debidos a cálculos erróneos, comparaciones incorrectas o flujo de control inadecuado.



Las pruebas de unidad por lo general se consideran como adjuntas al paso de codificación. El diseño de las pruebas de unidad puede ocurrir antes de comenzar la codificación o después de generar el código fuente. La revisión de la información del diseño proporciona una guía para establecer casos de prueba que es probable que descubran errores en cada una de las categorías analizadas anteriormente. Cada caso de prueba debe acoplarse con un conjunto de resultados esperados.

Las pruebas de unidad se simplifican cuando se diseña un componente con alta cohesión. Cuando un componente aborda una sola función, el número de casos de prueba se reduce y los errores pueden predecirse y descubrirse con mayor facilidad.

Pruebas de Integración

Las pruebas de integración son una técnica sistemática para construir la arquitectura del software mientras se llevan a cabo pruebas para descubrir errores asociados con la interfaz. El objetivo es tomar los componentes probados de manera individual y construir una estructura de programa que se haya dictado por diseño.

Con frecuencia existe una tendencia a intentar la integración no incremental, es decir, a construir el programa usando un enfoque de big bang. Todos los componentes se combinan por adelantado. Todo el programa se prueba como un todo. ¡Y usualmente resulta el caos! Se descubre un conjunto de errores. La corrección se dificulta pues el aislamiento de las causas se complica por la vasta extensión de todo el programa. Una vez corregidos estos errores, otros nuevos aparecen y el proceso continúa en un bucle aparentemente interminable.

Integración descendente.

La prueba de integración descendente es un enfoque incremental a la construcción de la arquitectura de software. Los módulos se integran al moverse hacia abajo a través de la jerarquía de control, comenzando con el módulo de control principal (programa principal). Los módulos subordinados al módulo de control principal se incorporan en la estructura en una forma de primero en profundidad o primero en anchura.

El proceso de integración se realiza en una serie de cinco pasos:

1. El módulo de control principal se usa como un controlador de prueba y los representantes (stubs) se sustituyen con todos los componentes directamente subordinados al módulo de control principal.
2. Dependiendo del enfoque de integración seleccionado (es decir, primero en profundidad o anchura), los representantes subordinados se sustituyen uno a la vez con componentes reales.
3. Las pruebas se llevan a cabo conforme se integra cada componente.
4. Al completar cada conjunto de pruebas, otro representante se sustituye con el componente real.
5. Las pruebas de regresión (que se analizan más adelante en esta sección) pueden realizarse para asegurar que no se introdujeron nuevos errores.



El proceso continúa desde el paso 2 hasta que se construye toda la estructura del sistema informático.

Integración ascendente.

La prueba de integración ascendente, como su nombre implica, comienza la construcción y la prueba con módulos atómicos (es decir, componentes en los niveles inferiores dentro de la estructura del programa). Puesto que los componentes se integran de abajo hacia arriba, la funcionalidad que proporcionan los componentes subordinados en determinado nivel siempre está disponible y se elimina la necesidad de representantes (stubs). Una estrategia de integración ascendente puede implementarse con los siguientes pasos:

1. Los componentes en el nivel inferior se combinan en grupos (en ocasiones llamados construcciones o builds) que realizan una subfunción de software específica.
2. Se escribe un controlador (un programa de control para pruebas) a fin de coordinar la entrada y salida de casos de prueba.
3. Se prueba el grupo.
4. Los controladores se remueven y los grupos se combinan moviéndolos hacia arriba en la estructura del programa

Prueba de regresión.

Cada vez que se agrega un nuevo módulo como parte de las pruebas de integración, el software cambia. Se establecen nuevas rutas de flujo de datos, ocurren nuevas operaciones de entrada/salida y se invoca nueva lógica de control. Dichos cambios pueden causar problemas con las funciones que anteriormente trabajaban sin fallas. En el contexto de una estrategia de prueba de integración, la prueba de regresión es la nueva ejecución de algún subconjunto de pruebas que ya se realizaron a fin de asegurar que los cambios no propagaron efectos colaterales no deseados.

En un contexto más amplio, las pruebas exitosas (de cualquier tipo) dan como resultado el descubrimiento de errores, y los errores deben corregirse. Siempre que se corrige el software, cambia algún aspecto de la configuración del software (el programa, su documentación o los datos que sustenta). Las pruebas de regresión ayudan a garantizar que los cambios (debidos a pruebas o por otras razones) no introducen comportamiento no planeado o errores adicionales.

Las pruebas de regresión se pueden realizar manualmente, al volver a ejecutar un subconjunto de todos los casos de prueba o usando herramientas de captura/reproducción automatizadas. Las herramientas de captura/reproducción permiten al ingeniero de software capturar casos de prueba y resultados para una posterior reproducción y comparación. La suite de prueba de regresión (el subconjunto de pruebas que se va a ejecutar) contiene tres clases diferentes de casos de prueba:

1. Una muestra representativa de pruebas que ejercitará todas las funciones de software.



2. Pruebas adicionales que se enfocan en las funciones del software que probablemente resulten afectadas por el cambio.
3. Pruebas que se enfocan en los componentes del software que cambiaron.

Conforme avanza la prueba de integración, el número de pruebas de regresión puede volverse muy grande. Por tanto, la suite de pruebas de regresión debe diseñarse para incluir solamente aquellas que aborden una o más clases de errores en cada una de las funciones del programa principal. Es impráctico e ineficiente volver a ejecutar toda prueba para cada función del programa cada vez que ocurre un cambio.

Pruebas de Validación

Las pruebas de validación comienzan en la culminación de las pruebas de integración, cuando se ejercitaron componentes individuales, el software está completamente ensamblado como un paquete y los errores de interfaz se descubrieron y corrigieron. En el nivel de validación o de sistema, desaparece la distinción entre software convencional, software orientado a objetos y webapps. Las pruebas se enfocan en las acciones visibles para el usuario y las salidas del sistema reconocibles por el usuario.

La validación puede definirse en muchas formas, pero una definición simple (aunque dura) es que la validación es exitosa cuando el software funciona en una forma que cumpla con las expectativas razonables del cliente. Si se desarrolló una Especificación de requerimientos de software, en ella se describen todos los atributos del software visibles para el usuario; contiene una sección de Criterios de validación que forman la base para un enfoque de pruebas de validación.

Criterios de pruebas de validación

La validación del software se logra a través de una serie de pruebas que demuestran conformidad con los requerimientos. Un plan de prueba subraya las clases de pruebas que se van a realizar y un procedimiento de prueba define casos de prueba específicos que se diseñan para garantizar que: se satisfacen todos los requerimientos de funcionamiento, se logran todas las características de comportamiento, todo el contenido es preciso y se presenta de manera adecuada, se logran todos los requerimientos de rendimiento, la documentación es correcta y se satisfacen la facilidad de uso y otros requerimientos (por ejemplo, transportabilidad, compatibilidad, recuperación de error, mantenimiento).

Después de realizar cada caso de prueba de validación, existen dos posibles condiciones:

1. La característica de función o rendimiento se conforma de acuerdo con las especificaciones y se acepta.
2. Se descubre una desviación de la especificación y se crea una lista de deficiencias. Las desviaciones o errores descubiertos en esta etapa en un proyecto rara vez pueden corregirse antes de la entrega calendarizada. Con frecuencia es necesario negociar con el cliente para establecer un método para resolver deficiencias.



Revisión de la configuración

Un elemento importante del proceso de validación es una revisión de la configuración. La intención de la revisión es garantizar que todos los elementos de la configuración del software se desarrollaron de manera adecuada, y que se cataloga y se tiene el detalle necesario para reforzar las actividades de apoyo.

Pruebas alfa y beta

Si el software se desarrolla como un producto que va a ser usado por muchos clientes, no es práctico realizar pruebas de aceptación formales con cada uno de ellos. La mayoría de los constructores de productos de software usan un proceso llamado prueba alfa y prueba beta para descubrir errores que al parecer sólo el usuario final es capaz de encontrar. La prueba alfa se lleva a cabo en el sitio del desarrollador por un grupo representativo de usuarios finales. El software se usa en un escenario natural con el desarrollador “mirando sobre el hombro” de los usuarios y registrando los errores y problemas de uso. Las pruebas alfa se realizan en un ambiente controlado.

La prueba beta se realiza en uno o más sitios del usuario final. A diferencia de la prueba alfa, por lo general el desarrollador no está presente. Por tanto, la prueba beta es una aplicación “en vivo” del software en un ambiente que no puede controlar el desarrollador. El cliente registra todos los problemas (reales o imaginarios) que se encuentran durante la prueba beta y los reporta al desarrollador periódicamente. Como resultado de los problemas reportados durante las pruebas beta, es posible hacer modificaciones y luego preparar la liberación del producto de software a toda la base de clientes.

En ocasiones se realiza una variación de la prueba beta, llamada prueba de aceptación del cliente, cuando el software se entrega a un cliente bajo contrato. El cliente realiza una serie de pruebas específicas con la intención de descubrir errores antes de aceptar el software del desarrollador. En algunos casos (por ejemplo, un gran corporativo o sistema gubernamental) la prueba de aceptación puede ser muy formal y abarcar muchos días o incluso semanas de prueba.

Pruebas de Sistema

Estas pruebas quedan fuera del ámbito del proceso de software y no se llevan a cabo exclusivamente por parte de ingenieros de software. Sin embargo, los pasos que se toman durante el diseño y la prueba del software pueden mejorar enormemente la probabilidad de integración exitosa del software en el sistema más grande.

Un problema clásico en la prueba del sistema es el “dedo acusador”. Esto ocurre cuando se descubre un error y los desarrolladores de diferentes elementos del sistema se culpan unos a otros por el problema. En lugar de abandonarse a tal sinsentido, deben anticiparse los potenciales problemas de interfaz y: 1) diseñar rutas de manejo de error que prueben toda la



información proveniente de otros elementos del sistema, 2) realizar una serie de pruebas que simulen los datos malos u otros errores potenciales en la interfaz del software, 3) registrar los resultados de las pruebas para usar como “evidencia” si ocurre el dedo acusador, y 4) participar en planificación y diseño de pruebas del sistema para garantizar que el software se prueba de manera adecuada.

En realidad, la prueba del sistema es una serie de diferentes pruebas cuyo propósito principal es ejercitar por completo el sistema basado en computadora. Aunque cada prueba tenga un propósito diferente, todo él funciona para verificar que los elementos del sistema se hayan integrado de manera adecuada y que se realicen las funciones asignadas. En las secciones que siguen se estudian los tipos de pruebas del sistema que valen la pena para los sistemas basados en software.

Pruebas No Funcionales

Estas pruebas son parte de las pruebas del sistema porque están orientadas completar la funcionalidad que debe de tener todo sistema informático. Algunas de las pruebas de este tipo son

- Pruebas de recuperación
- Pruebas de seguridad
- Pruebas de esfuerzo
- Pruebas de rendimiento
- Pruebas de despliegue
- Pruebas de mantenibilidad
- Pruebas de usabilidad

Pruebas de recuperación

Muchos sistemas basados en computadora deben recuperarse de fallas y reanudar el procesamiento con poco o ningún tiempo de inactividad. En algunos casos, un sistema debe ser tolerante a las fallas, es decir, las fallas del procesamiento no deben causar el cese del funcionamiento del sistema global. En otros casos, la falla de un sistema debe corregirse dentro de un periodo de tiempo específico u ocurrirán severos daños económicos.

La recuperación es una prueba del sistema que fuerza al software a fallar en varias formas y que verifica que la recuperación se realice de manera adecuada. Si la recuperación es automática (realizada por el sistema en sí), se evalúa el reinicio, los mecanismos de puntos de verificación, la recuperación de datos y la reanudación para correcciones. Si la recuperación requiere intervención humana, se evalúa el tiempo medio de reparación (TMR) para determinar si está dentro de límites aceptables.



Pruebas de seguridad

Cualquier sistema basado en computadora que gestione información sensible o cause acciones que puedan dañar (o beneficiar) de manera inadecuada a individuos es un blanco de penetración inadecuada o ilegal. La penetración abarca un amplio rango de actividades: hackers que intentan penetrar en los sistemas por deporte, empleados resentidos que intentan penetrar por venganza, individuos deshonestos que intentan penetrar para obtener ganancia personal ilícita.

La prueba de seguridad intenta verificar que los mecanismos de protección que se construyen en un sistema en realidad lo protegerán de cualquier penetración impropia. Para citar a Beizar “La seguridad del sistema debe, desde luego, probarse para ser invulnerable ante ataques frontales; pero también debe probarse su invulnerabilidad contra ataques laterales y traseros.”

Durante la prueba de seguridad, quien realiza la prueba juega el papel del individuo que desea penetrar al sistema. ¡Cualquier cosa vale! Quien realice la prueba puede intentar adquirir contraseñas por medios administrativos externos; puede atacar el sistema con software a la medida diseñado para romper cualquier defensa que se haya construido; puede abrumar al sistema, y por tanto negar el servicio a los demás; puede causar a propósito errores del sistema con la esperanza de penetrar durante la recuperación; puede navegar a través de datos inseguros para encontrar la llave de la entrada al sistema.

Con los suficientes tiempo y recursos, las buenas pruebas de seguridad a final de cuentas penetran en el sistema. El papel del diseñador de sistemas es hacer que el costo de la penetración sea mayor que el valor de la información que se obtendrá.

Pruebas de esfuerzo

Los primeros pasos de la prueba del software dieron como resultado una evaluación extensa de las funciones y el rendimiento normales del programa. Las pruebas de esfuerzo se diseñan para enfrentar los programas con situaciones anormales.

La prueba de esfuerzo ejecuta un sistema en forma que demanda recursos en cantidad, frecuencia o volumen anormales. Por ejemplo, pueden 1) diseñarse pruebas especiales que generen diez interrupciones por segundo, cuando una o dos es la tasa promedio, (2) aumentarse las tasas de entrada de datos en un orden de magnitud para determinar cómo responderán las funciones de entrada, 3) ejecutarse casos de prueba que requieran memoria máxima y otros recursos, 4) diseñarse casos de prueba que puedan causar thrashing (que es un quebranto del sistema por hiperpaginación) en un sistema operativo virtual, 5) crearse casos de prueba que puedan causar búsqueda excesiva por datos residentes en disco. En esencia, la persona que realiza la prueba intenta romper el programa.

Una variación de la prueba de esfuerzo es una técnica llamada prueba de sensibilidad. En algunas situaciones (la más común ocurre en algoritmos matemáticos), un rango muy pequeño de datos contenidos dentro de las fronteras de los datos válidos para un programa



puede causar procesamiento extremo, e incluso erróneo, o profunda degradación del rendimiento. La prueba de sensibilidad intenta descubrir combinaciones de datos dentro de clases de entrada válidas que puedan causar inestabilidad o procesamiento inadecuado.

Pruebas de rendimiento

Para sistemas en tiempo real y sistemas embebidos, el software que proporcione la función requerida, pero que no se adecue a los requerimientos de rendimiento, es inaceptable. La prueba de rendimiento se diseña para poner a prueba el rendimiento del software en tiempo de corrida, dentro del contexto de un sistema integrado. La prueba del rendimiento ocurre a lo largo de todos los pasos del proceso de prueba. Incluso en el nivel de unidad, puede accederse al rendimiento de un módulo individual conforme se realizan las pruebas. Sin embargo, no es sino hasta que todos los elementos del sistema están plenamente integrados cuando puede determinarse el verdadero rendimiento de un sistema.

Las pruebas de rendimiento con frecuencia se aparean con las pruebas de esfuerzo y por lo general requieren instrumentación de hardware y de software, es decir, con frecuencia es necesario medir la utilización de los recursos (por ejemplo, ciclos del procesador) en forma meticulosa. La instrumentación externa puede monitorear intervalos de ejecución y eventos de registro (por ejemplo, interrupciones) conforme ocurren, y los muestreos del estado de la máquina de manera regular. Con la instrumentación de un sistema, la persona que realiza la prueba puede descubrir situaciones que conduzcan a degradación y posibles fallas del sistema.

Pruebas de despliegue

En muchos casos, el software debe ejecutarse en varias plataformas y bajo más de un entorno de sistema operativo. La prueba de despliegue, en ocasiones llamada prueba de configuración, ejercita el software en cada entorno en el que debe operar. Además, examina todos los procedimientos de instalación y el software de instalación especializado (por ejemplo, “instaladores”) que usarán los clientes, así como toda la documentación que se usará para introducir el software a los usuarios finales.

Pruebas de Mantenibilidad

Es la evaluación de qué tan fácil es realizar el mantenimiento de la aplicación. Se evalúan aspectos como si los programadores han seguido buenas prácticas de programación, si se cumplió con los estándares establecidos por el equipo de desarrollo

Para realizar esta prueba deben evaluarse la forma en que está implementada la aplicación, siguiendo buenas prácticas de ingeniería de software. Es decir, que se estén siguiendo los patrones recomendados de ingeniería de software y que no se estén introduciendo



inadvertidamente anti patrones, esto es, que no se estén cometiendo errores comunes de programación.

Pruebas de Usabilidad

Las pruebas de usabilidad consisten en reunir a los clientes de la aplicación y solicitarles que lleven a cabo las tareas para las cuales fue diseñada, en tanto el equipo de diseño, desarrollo y otros involucrados toman nota de la interacción, particularmente de los errores y dificultades con las que se encuentren los usuarios.

En las pruebas de usabilidad, los testers se enfocan en validar que tan fácil de usar es una determinada aplicación.

Las características evaluadas en la usabilidad incluyen:

- Facilidad de aprendizaje.
- Eficiencia
- Memorización
- Errores
- Satisfacción



Bibliografía

- Pressman, R. S. (2010). *Ingeniería De Software. Un Enfoque Práctico (7.a ed.)*. MCGRAW HILL EDUCATION.
- it-Mentor. (2009). *Capacitación y guía para el desarrollo de software [Material]*. Materias fi.uba.ar. <http://materias.fi.uba.ar/7548/PruebasSoftware.pdf>
- EcuRed. (2019). *Pruebas de software - EcuRed*. https://www.ecured.cu/Pruebas_de_software. Accedida en Junio 2020
- López, F. J. G. (2019, 18 diciembre). *Testing en un mundo Agile*. Adictos al trabajo. <https://www.adictosaltrabajo.com/2019/12/18/testing-en-un-mundo-agile/>. Accedida en Junio 2020.
- R. (2018, julio). *Agile Testing Methodology - Methods, Principles & Advantages*. ReQtest. <https://reqtest.com/testing-blog/agile-testing-principles-methods-advantages/>. Accedida en Junio 2020.
- PMOinformatica.com. (2015, 18 marzo). *Qué es el Agile Testing y cuáles son sus principios y estrategias*. La Oficina de Proyectos de Informática. <http://www.pmoinformatica.com/2015/03/que-es-el-agile-testing.html> Accedido en Junio 2020
- blogger.com. (2016, 1 agosto). *10 tipos de pruebas no funcionales*. La Oficina de Proyectos de Informática. <http://www.pmoinformatica.com/2016/07/tipos-pruebas-no-funcionales.html> Accedido en Junio 2020